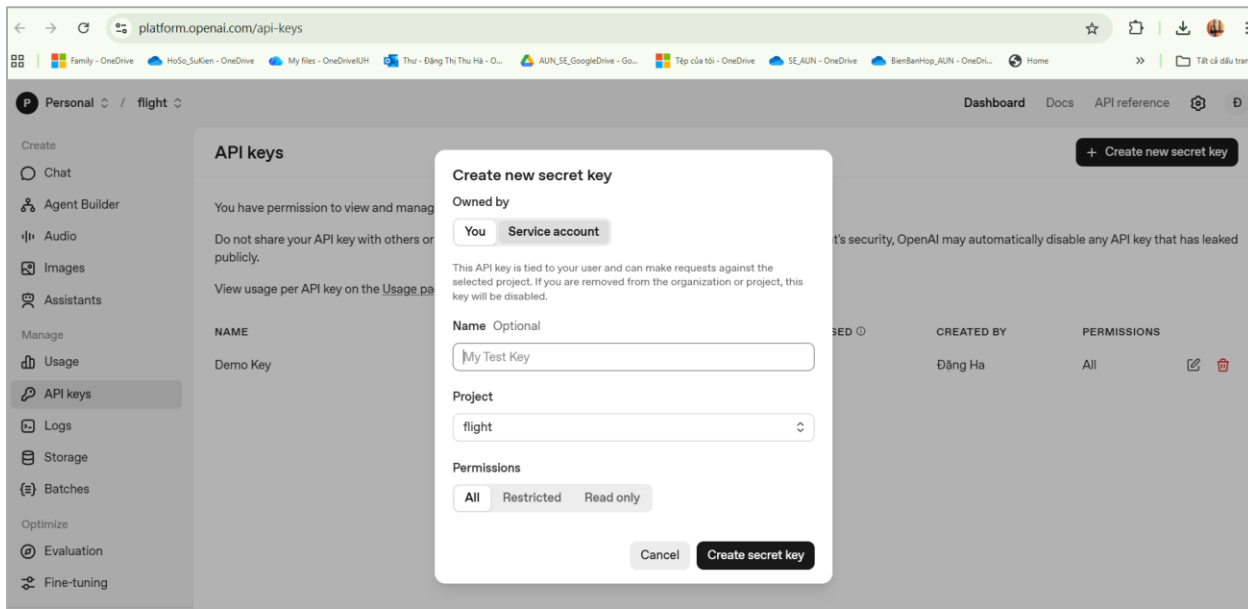


# Spring AI:

Lấy **API Key** trên các nền tảng AI phổ biến:

## I. Đối với OpenAI (ChatGPT, GPT-4)

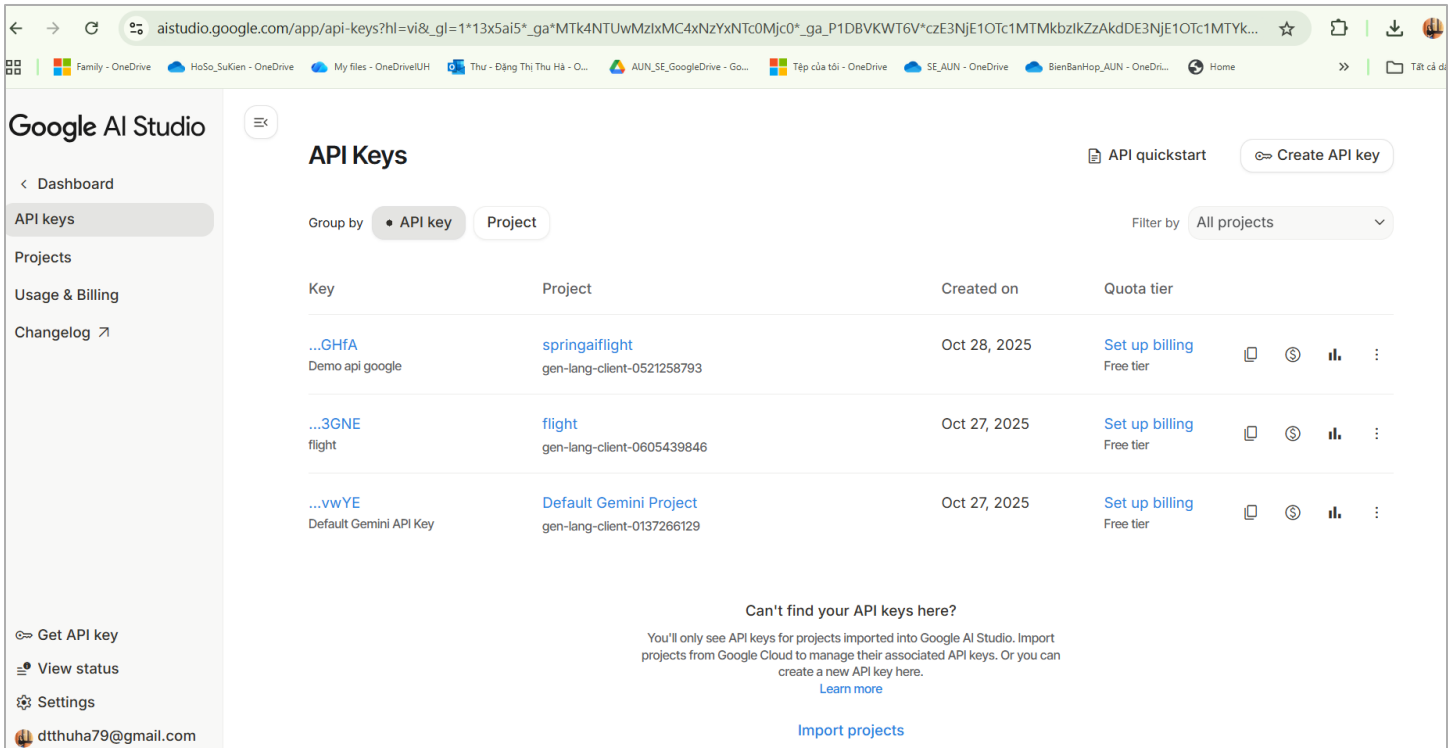


1. **Truy cập** [API keys của OpenAI](#).
2. **Đăng nhập/Đăng ký:** Đăng nhập vào tài khoản OpenAI của bạn (hoặc đăng ký nếu chưa có).
3. **Tạo Khóa Mới:** Nhấp vào nút **"+ Create new secret key"** (Tạo khóa bí mật mới).
4. **Đặt Tên và Tạo:** Đặt tên cho khóa của bạn (ví dụ: "Spring-AI-App") rồi nhấp vào **"Create secret key"**.
5. **Sao chép: Quan trọng!** API Key sẽ chỉ hiển thị một lần. Hãy sao chép và lưu trữ nó ở nơi an toàn ngay lập tức.

---

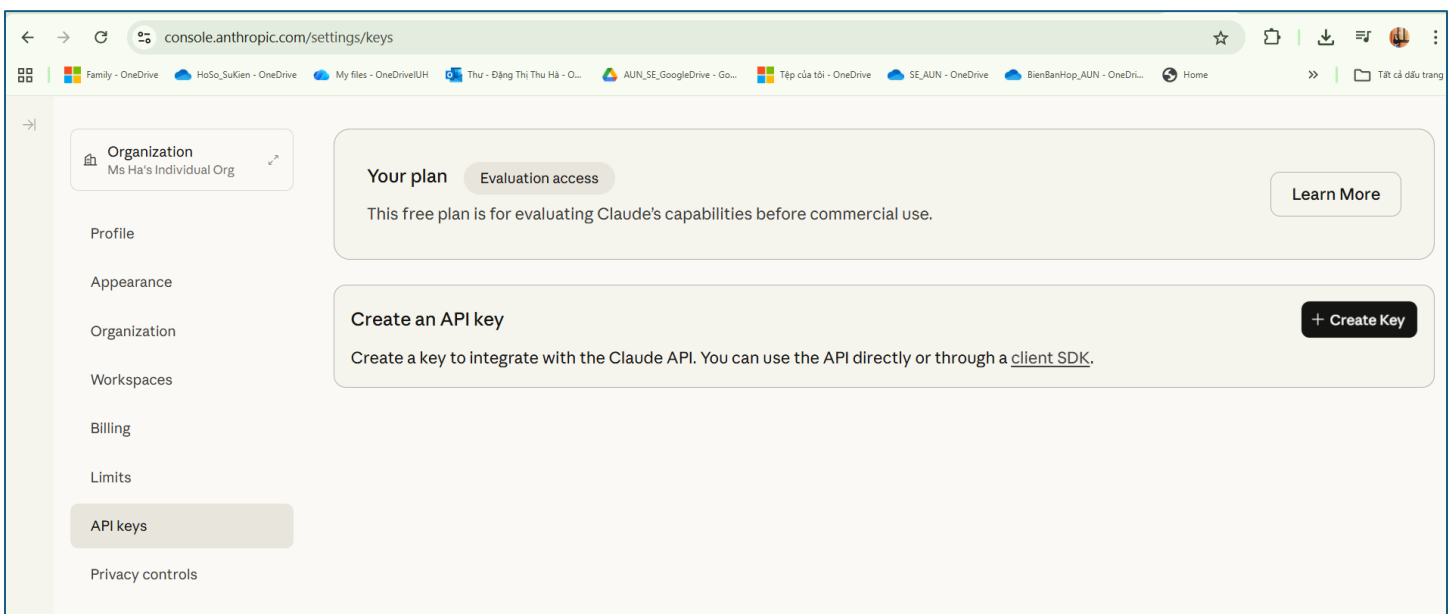
## II. Đối với Google (Gemini)

Google sử dụng **API Key** cho các dịch vụ AI tổng quát như Gemini thông qua **Google AI Studio** hoặc sử dụng **Service Account** cho các dịch vụ cấp doanh nghiệp (Vertex AI).



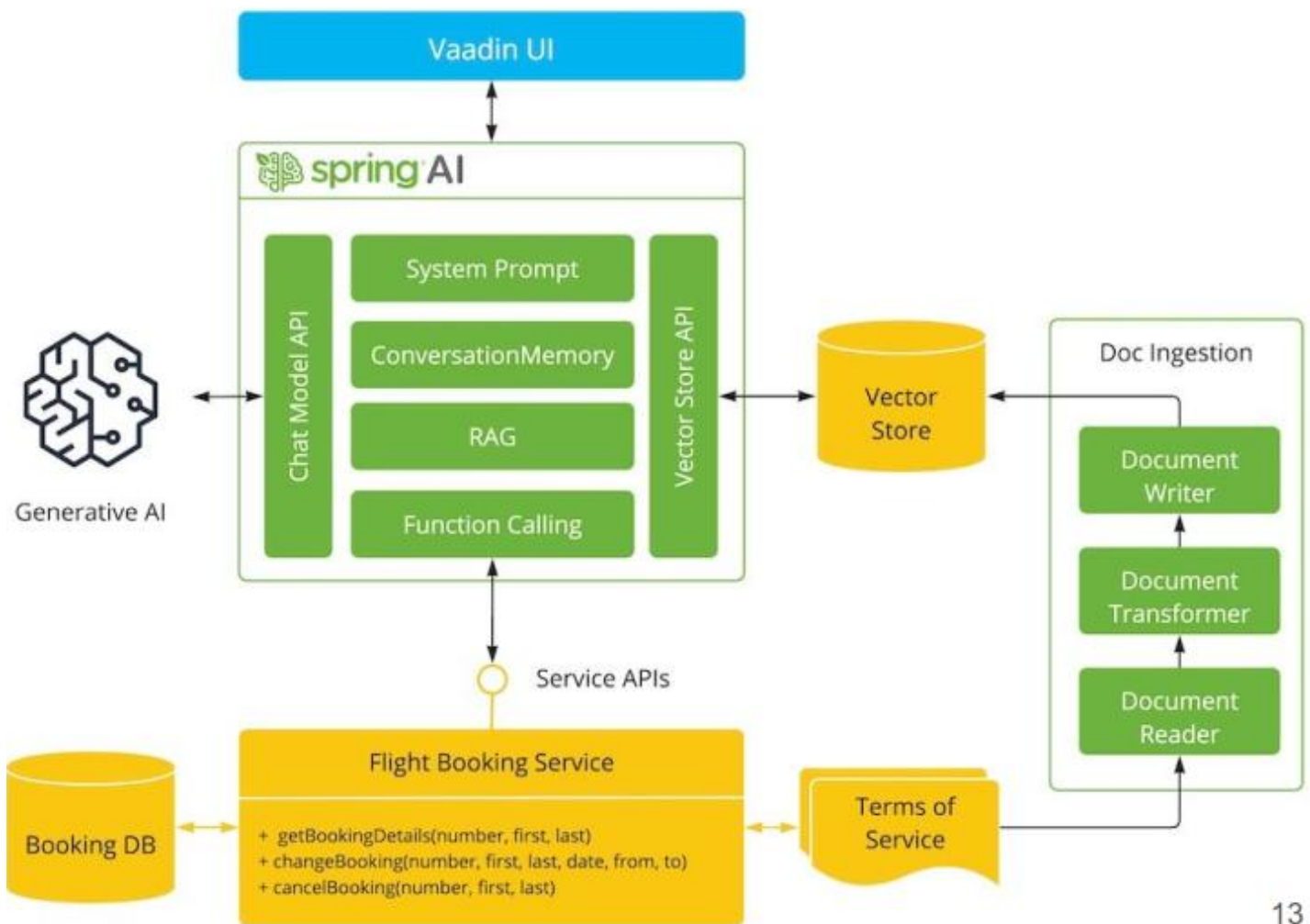
1. **Truy cập** [Google AI Studio](#).
2. **Đăng nhập:** Đăng nhập bằng tài khoản Google của bạn.
3. **Tạo Khóa:** Nhấp vào nút **"Create API key in Google AI Studio"**.
4. **Sao chép:** API Key của bạn sẽ được tạo và hiển thị. Sao chép và lưu lại.

### III. Đối với Anthropic (Claude)



1. **Truy cập** <https://console.anthropic.com/settings/keys>
2. **Tạo Khóa:** Nhấp vào **"Create Key"** và làm theo hướng dẫn để tạo và sao chép API Key.

#### IV. Xây dựng một project hoàn chỉnh cho hệ thống Flight Booking Assistant sử dụng Spring Boot + Spring DataJPa + Spring AI + RAG + Function Calling



## Kiến trúc của project

```
spring-ai-flight-booking-assistant/  
├── src/  
│   ├── main/  
│   │   ├── java/com/dang/flightai/  
│   │   │   ├── controller/  
│   │   │   │   └── ChatController.java  
│   │   │   ├── service/  
│   │   │   │   ├── AiService.java  
│   │   │   │   └── BookingService.java  
│   │   │   ├── model/  
│   │   │   │   ├── BookingDetails.java  
│   │   │   │   └── FlightAiApplication.java  
│   │   └── resources/  
│   │       ├── application.yml  
│   │       └── documents/  
│   │           └── terms-of-service.txt  
├── .env  
├── pom.xml  
└── README.md
```

## Các file gợi ý:

application.yml

```
server:  
  port: 8090  
  
spring:  
  ai:  
    openai:  
      api-key: ${OPENAI_API_KEY}  
    chat:  
      options:  
        model: gpt-4  
        temperature: 0.8  
        max-tokens: 1000  
  
    vectorstore:  
      qdrant:  
        endpoint: http://localhost:6333  
    anthropic:  
      api-key: ${ANTHROPIC_API_KEY}  
  
application:  
  name: spring-ai-flight
```

 .env

Env

 Copy

OPENAI\_API\_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

## Các dependencies cần thiết:

1. *Spring Boot Starter*
2. *Spring Boot DevTools (tự động reload khi code thay đổi)*
3. *Lombok (giảm boilerplate code)*
4. *Spring Boot Test (viết unit test)*
5. *Spring Web (REST API)*
6. *Spring AI Core + OpenAI*
7. *Vector Store (Qdrant hoặc Chroma)*
8. *Spring AI Advisors (RAG, Tool Calling)*
9. *Spring Boot Configuration Processor (hỗ trợ IDE)*
10. *Vaadin UI (giao diện người dùng)*
11. *SnakeYAML (đọc file YAML – cấu hình application.yml)*

## ChatController.java

```
@RestController
@RequestMapping("/chat")
public class ChatController {
    @Autowired
    private AiService aiService;
    @PostMapping
    public ResponseEntity<String> chat(@RequestBody String userInput)
    {
        String response = aiService.processUserInput(userInput);
        return ResponseEntity.ok(response);
    }
}
```

## AIService.class

@Service

```
public class AIService {
```

    @Autowired

```
    private ChatClient chatClient;
```

    @Autowired

```
    private Retriever retriever;
```

```
    public String processUserInput(String input) {
```

```
        List<Document> contextDocs = retriever.retrieve(input);
```

```
        String context = contextDocs.stream()
```

```
            .map(Document::getContent)
```

```
            .collect(Collectors.joining("\n"));
```

```
        String prompt = ""
```

```
            You are a flight assistant.
```

```
            Context: %s
```

```
            User question: %s
```

```
        "".formatted(context, input);
```

```
        return chatClient.call(prompt).getResult().getOutput().getContent();
```

```
    }
```

```
}
```

## BookingService.class

@Service

```
public class BookingService {  
  
    public BookingDetails getBookingDetails(String number, String first, String last) {  
        return new BookingDetails(number, first, last, "Hanoi", "Tokyo", "2025-11-05");  
    }  
  
    public String changeBooking(...) { /* logic */ }  
  
    public String cancelBooking(...) { /* logic */ }  
}
```

## BookingDetails.class

```
public class BookingDetails {  
    private String number;  
    private String firstName;  
    private String lastName;  
    private String from;  
    private String to;  
    private String date;  
  
    // Constructor, getters, setters  
}
```





terms-of-service.txt

Copy

To cancel a flight, provide your booking number, full name, and  
Changes must be made at least 24 hours before departure.



## RAG + VectorStore

- Ingest tài liệu:

Java

Copy

```
vectorStore.add(List.of(new Document("Cancellation policy ...",
```

- Truy xuất:

Java

Copy

```
Retriever retriever = new VectorStoreRetriever(vectorStore);  
List<Document> context = retriever.retrieve("How to cancel my
```



## Function Calling

Java

Copy

```
@Function  
public BookingDetails getBookingDetails(String number, String  
    return bookingService.getBookingDetails(number, first, las  
}
```

 Copy

```
├── FlightAiApplication.java
├── controller/
│   └── ChatController.java
├── service/
│   └── AiService.java
└── ui/
    └── MainView.java
```

### **MainView.java**

```
@Route("")
@CssImport("./styles/styles.css")
public class MainView extends VerticalLayout {

    private final RestTemplate restTemplate = new RestTemplate();
    private final TextArea inputArea = new TextArea("Nhập câu hỏi:");
    private final Button sendButton = new Button("Gửi");
    private final TextArea responseArea = new TextArea("Phản hồi từ AI:");

    public MainView() {
        inputArea.setWidthFull();
        responseArea.setWidthFull();
        responseArea.setReadOnly(true);

        sendButton.addClickListener(e -> {
            String prompt = inputArea.getValue();

            String response = restTemplate.postForObject("http://localhost:8080/chat", prompt,
String.class);
            responseArea.setValue(response);
        });

        add(inputArea, sendButton, responseArea);
        setPadding(true);
        setSpacing(true);
    }
}
```